FeedInsight – AI Product Intelligence & Backlog Automation

Project Title: FeedInsight – AI Product Intelligence & Backlog Automation

Team Name: FeedInsight Team

Target Industry: Tech

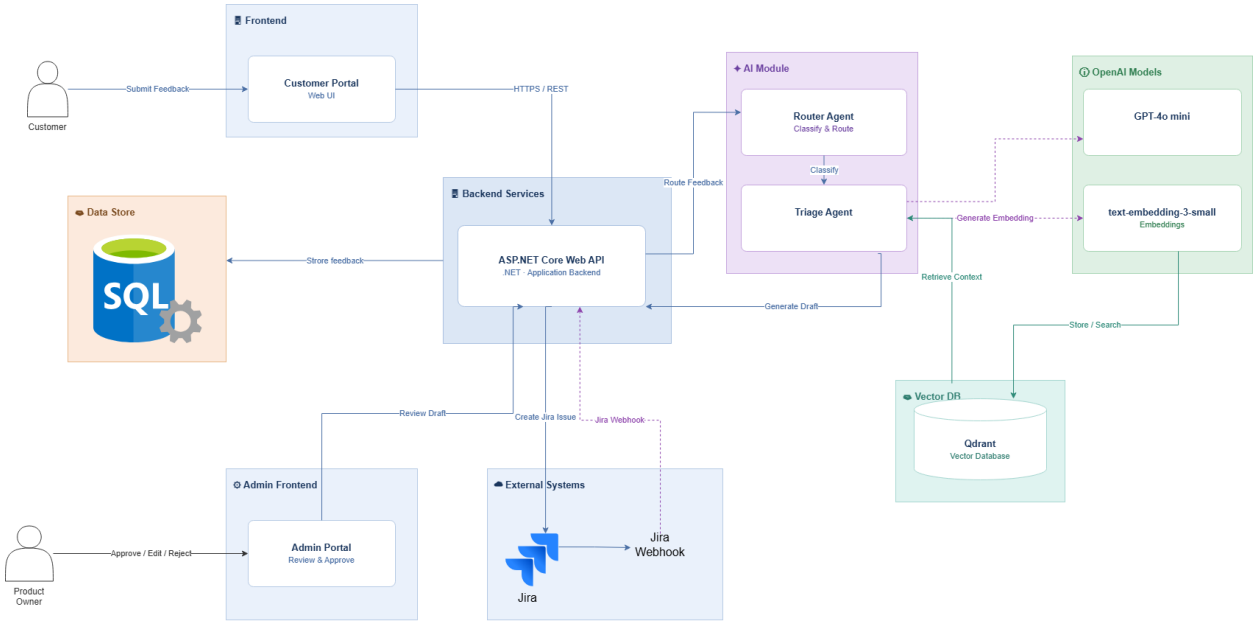
Team Members:

1. Abdelrahman Reda Mohammed
2. Abdelrahman Nasr Aql
3. Alaa Essam Ali
4. Mostafa Shaaban Mohamed
5. Ahmed Essam Hamdy
6. Ali Azouz Ali
7. Yousef Hanna Anees

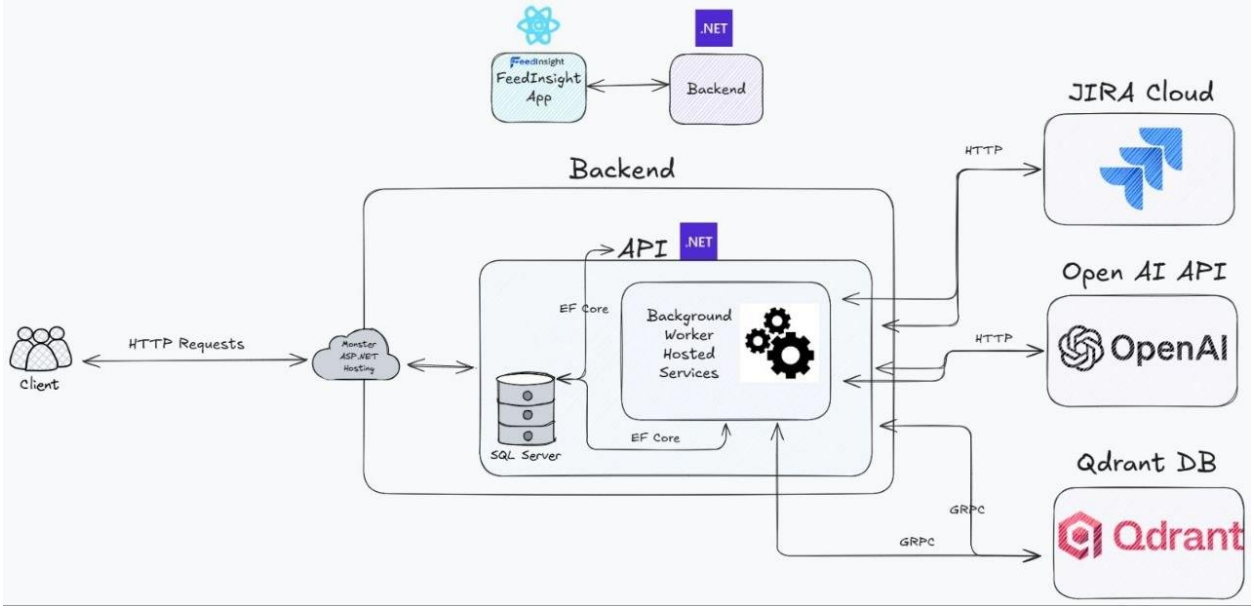
Sprint 0 – Project Design & Setup Sprint

1. System Design

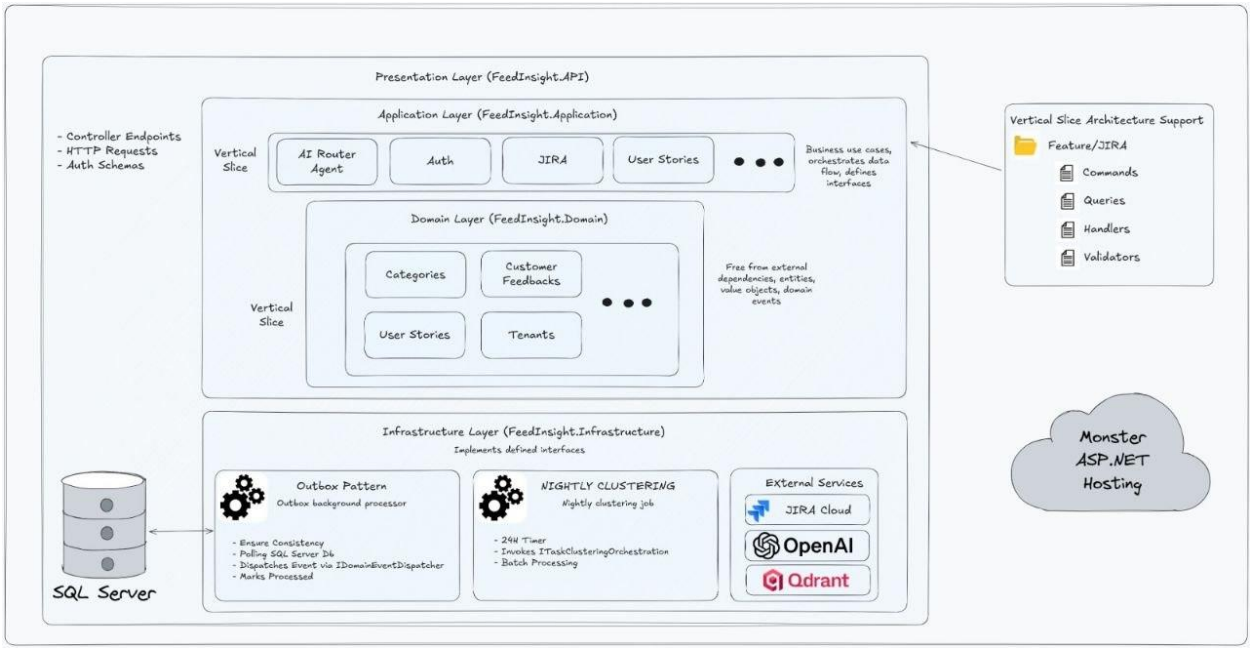
1.1 system Architecture



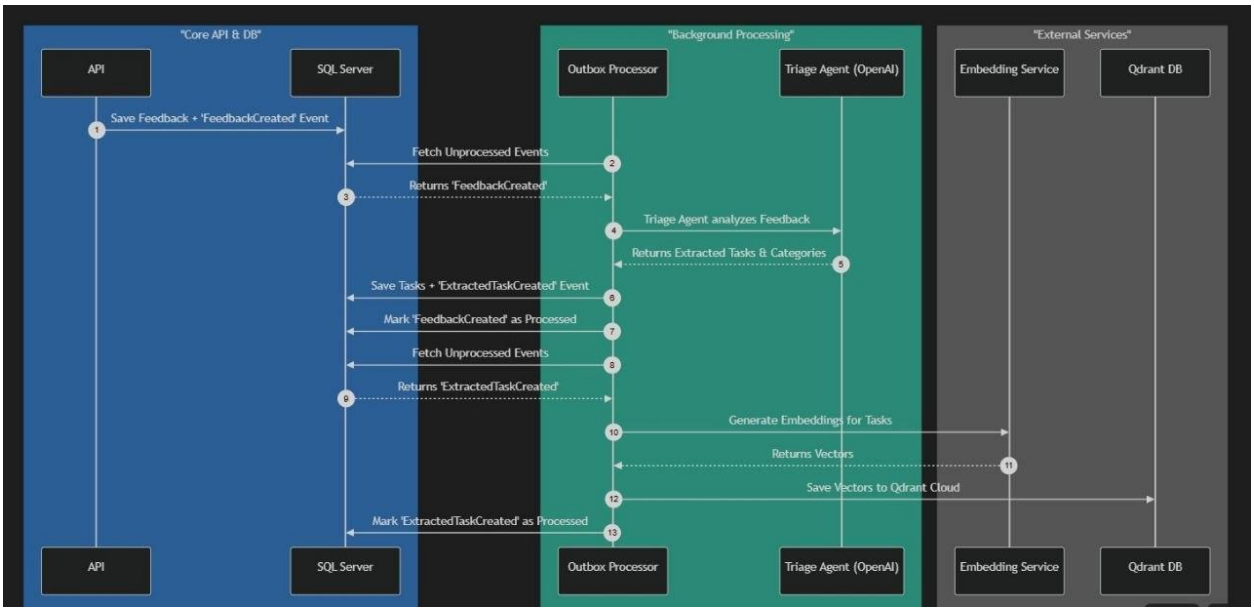
1.2 Container Diagram

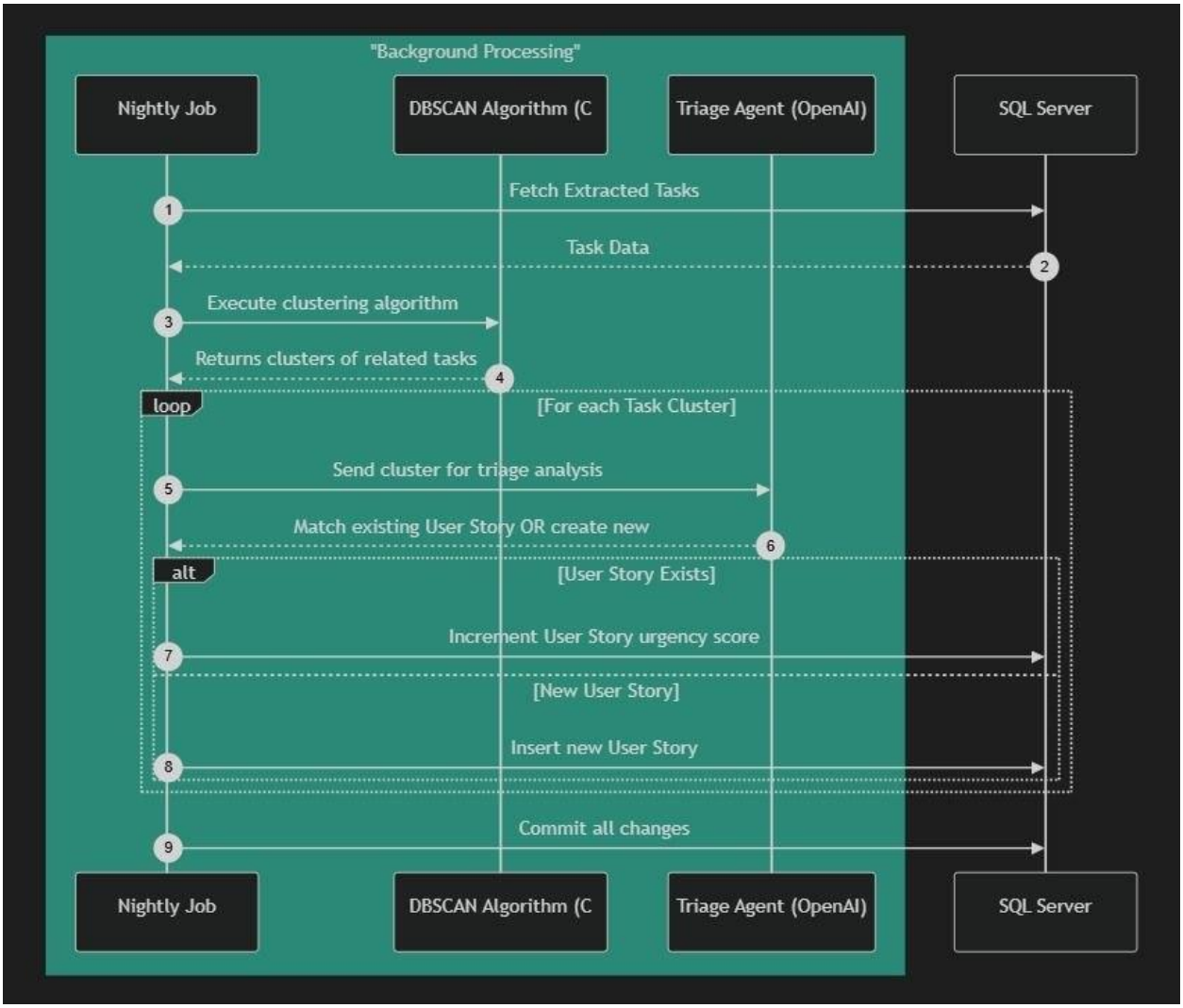


1.3 Clean Architecture



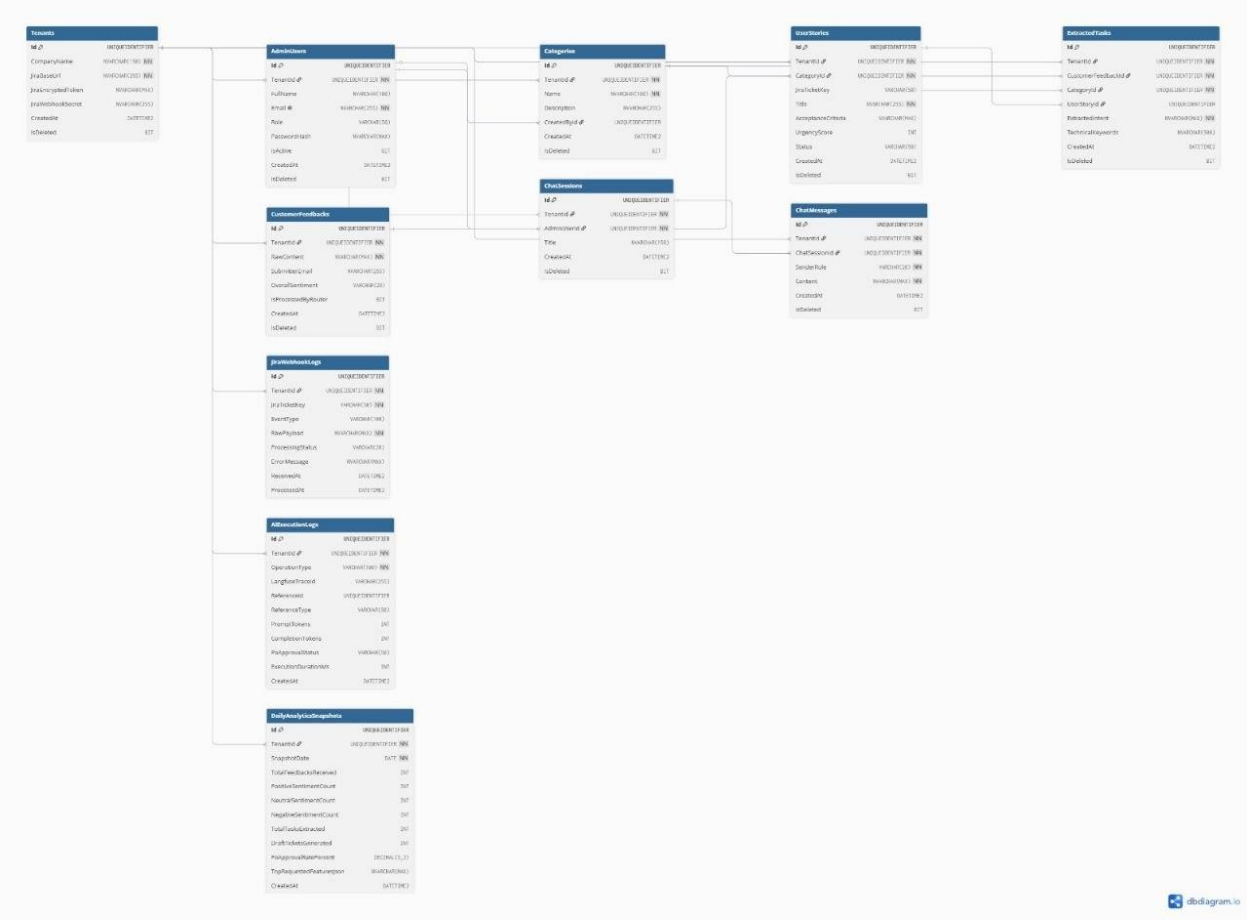
1.4 Sequence diagram


"Core API & DB"
"Background Processing"
"External Services"
API
SQL Server
Outbox Processor
Triage Agent (OpenAI)
Embedding Service
Qdrant DB
Save Feedback + 'FeedbackCreated' Event
Fetch Unprocessed Events
Returns 'FeedbackCreated'
Triage Agent analyzes Feedback
Returns Extracted Tasks & Categories
Save Tasks + 'ExtractedTaskCreated' Event
Mark 'FeedbackCreated' as Processed
Fetch Unprocessed Events
Returns 'ExtractedTaskCreated'
Generate Embeddings for Tasks
Returns Vectors
Save Vectors to Qdrant Cloud
Mark 'ExtractedTaskCreated' as Processed
API
SQL Server
Outbox Processor
Triage Agent (OpenAI)
Embedding Service
Qdrant DB


"Background Processing"
Nightly Job
DBSCAN Algorithm (C
Triage Agent (OpenAI)
SQL Server
Fetch Extracted Tasks
Task Data
Execute clustering algorithm
Returns clusters of related tasks
loop
[For each Task Cluster]
Send cluster for triage analysis
Match existing User Story OR create new
alt
[User Story Exists]
Increment User Story urgency score
[New User Story]
Insert new User Story
Commit all changes
Nightly Job
DBSCAN Algorithm (C
Triage Agent (OpenAI)
SQL Server

2. Database & API Design

2.1 ERD diagram



3. AI Design (AI Projects)

1. LLM

• Model: GPT-4o mini

• Configuration & Execution Strategy: The system utilizes OpenAI's GPT-4o mini configured with structured output JSON parsing modes to enforce strict payload adherence. We implement a Chain-of-Thought (CoT) prompting strategy to manage multi-intent customer feedback effectively. The prompt instructs the model to sequentially isolate distinct tasks, analyze their nature, and map user terminology into developer-ready contexts.

System Prompt Focus Areas:

• Intent Classification: Instructing the model to detect and isolate distinct engineering tasks found within compound customer inputs.

• Sentiment Analysis: Extracting the overarching and task-specific emotional tone (Positive, Neutral, Negative).

• Technical Translation: Converting colloquial customer complaints into specific technical domain keywords to optimize later semantic retrieval.

2. Agent Framework

• Framework: Microsoft Semantic Kernel

Architectural Boundaries & Guidelines:

• Router Agent: This agent operates at the live feedback routing pipeline. It is responsible for analyzing the raw text payloads received from the Customer Portal. If it detects multi-intents, it splits the payload into N isolated intents and assigns dynamic categories and keywords before storing them as Extracted Tasks SQL rows. It operates completely statelessly.

• Triage Agent: Operating downstream, this agent triggers the semantic K-NN search against the Qdrant database. Its strict boundary involves decision-making based on similarity scores. If a duplicate ticket match is found, it links the Extracted Task to an existing User Story ID. If it is a distinct problem, it commands the LLM to generate a new title alongside explicit "Given-When-Then" acceptance criteria, saving it with a 'Draft' status.

3. Vector Database

• **Database: Qdrant**

Technical Configuration Parameters:

• Semantic Chunking Strategy: The platform uses a single unified vector memory footprint. We map entities at a 1:1 ratio, meaning an entire Jira ticket or isolated feedback entity acts as an individual chunk.

• Vector Index Settings: We utilize native GUID string mappings for point IDs to ensure direct parity with the relational SQL Server database.

• Distance Metric & Search: Uses Semantic K-NN Search configured for Cosine Similarity to detect overlapping requests.

• Payload Filtering: Metadata payloads securely isolate data by embedding the TenantId to strictly enforce multi-tenant data boundaries at the vector level.

4. Embedding Model

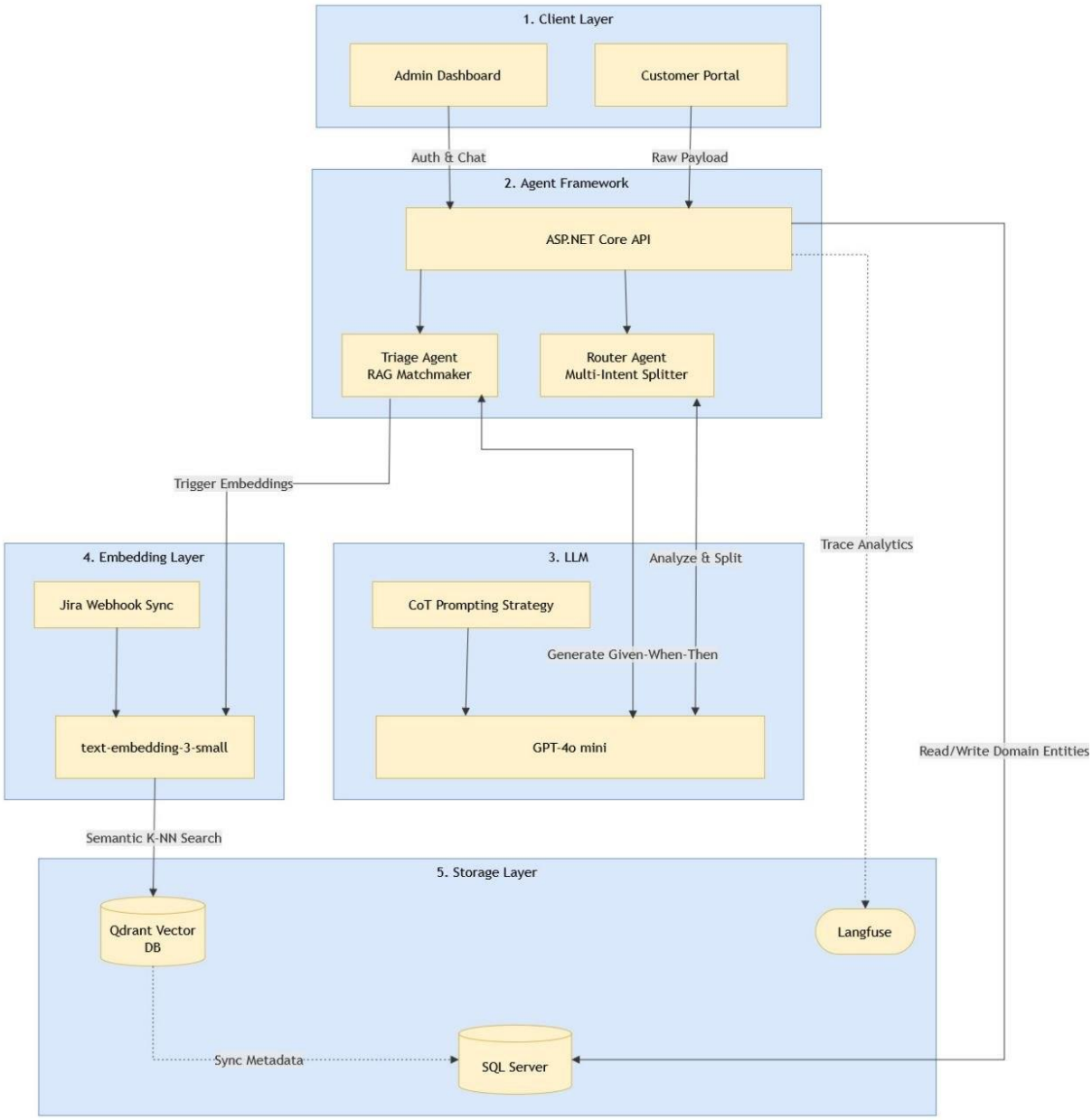
• Model: OpenAI text-embedding-3-small

• Technical Specifications: The model generates vectors utilizing 1536 dimensions for highly accurate semantic retrieval.

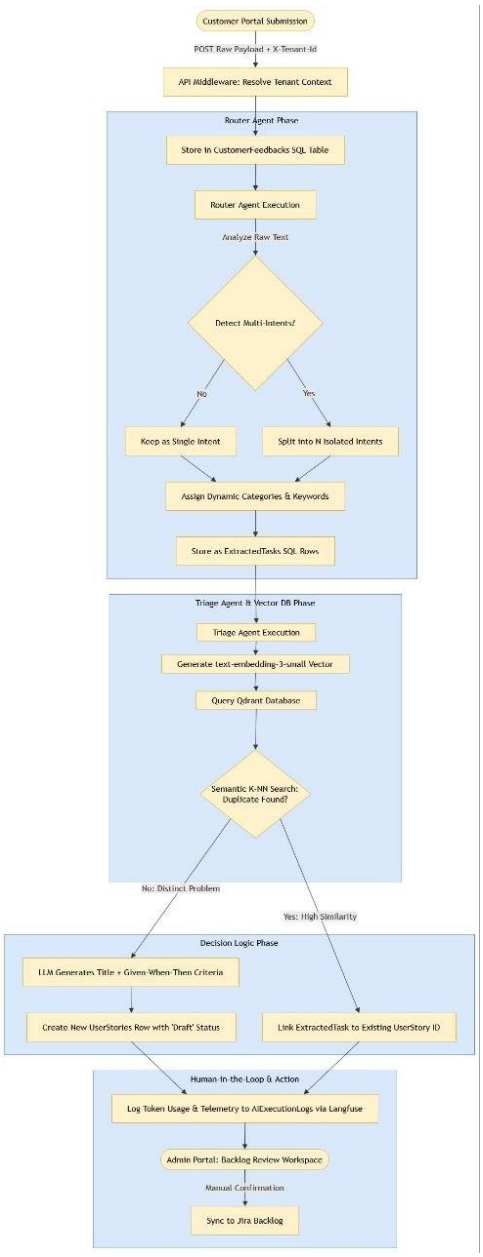
• Continuous Data Ingestion Pipeline: The Jira Sync RAG automates vectorization. When Atlassian Jira fires a webhook, the Web API validates the signature and writes the raw JSON payload to a pending SQL log. An asynchronous background sync service maps this payload to the UserStories domain entity, calls the embedding model, and pushes the new 1536-dimensional vector into Qdrant.

5. RAG/AI workflow

- **AI System Workflow**



- **RAG Workflow**



4. UI/UX Design

Figma Link: https://www.figma.com/design/zp1MA9J0CDq5sh4JAIsl4X/FeedInsight-AI-Assistant?node-id=0-1&t=61LZjgAI5VBRdFH9-1

5. Project Setup

Github link repo: https://github.com/orgs/FeedInsight/

6. Backlog Preparation

Epic 1: Customer Feedback Ingestion & Tenant Routing

#	User Story
1	As a Customer, I want to submit product feedback using natural, unstructured language via a public web form so that I can report my issues or ideas without navigating complex bug-reporting tools.
2	As a Product Owner, I want my public portal to automatically attach a specific Tenant ID to all incoming customer feedback so that my data is securely routed only to my company's database partition.

Epic 2: Multi-Tenancy & Access Control

#	User Story
1	As a Product Owner, I want to input and securely encrypt my Atlassian Jira API token and Webhook Secret so that the FeedInsight platform can safely communicate with my specific Jira instance.
2	As a Product Owner, I want to log into the Admin Portal using role-based authentication so that I can securely access my team's specific dashboard, categories, and backlog without seeing other companies' data.

Epic 3: AI Routing & Task Extraction (The Split)

#	User Story
1	As a Product Owner, I want to create and manage dynamic feedback categories (e.g., 'UI/UX', 'Core Bug') so that the AI Router Agent has predefined targets for sorting incoming tasks.
2	As a Product Owner, I want the AI Router Agent to automatically detect and split compound customer feedback into distinct, isolated tasks so that a single report containing multiple different issues (e.g., a bug and a feature request) can be triaged separately.

3	As a Product Owner, I want the AI Router Agent to automatically assign extracted tasks to my pre-defined dynamic categories so that incoming issues are immediately organized according to my team's workflow.

Epic 4: AI Triage & RAG Duplicate Detection (The Match)

#	**User Story**
1	As a Product Owner, I want the AI Triage Agent to automatically cross-reference extracted tasks against my existing Jira backlog using semantic vector search so that duplicate or overlapping issues are instantly flagged without manual reading.
2	As a Product Owner, I want the system to automatically link new duplicate tasks to the exact ID of an existing Jira ticket so that I can track how many unique customers are experiencing the exact same problem.
3	As a Product Owner, I want the AI Triage Agent to automatically generate a developer-ready draft User Story—complete with a title and explicit "Given-When-Then" Acceptance Criteria—for completely new issues so that I do not have to translate customer complaints into technical requirements from scratch.

Epic 5: Backlog Review Workspace & Jira Sync

#	**User Story**
1	As a Product Owner, I want to view a calculated Urgency Score for each backlog item (based on the volume of linked customer feedback and sentiment) so that I know exactly which issues are impacting users the most.
2	As a Product Owner, I want a dedicated workspace to manually review, edit, approve, or reject AI-generated draft stories so that I maintain absolute control over the requirements before they reach the engineering team.
3	As a Product Owner, I want the AI to extract technical keywords from colloquial customer language so that the system can perform highly accurate semantic searches against the engineering backlog.
4	As a Product Owner, I want to click an "Approve & Sync" button to push a finalized draft story directly to our active Jira instance so that developers can pull the new ticket into their active sprint immediately.
5	As a Product Owner, I want FeedInsight to listen to Jira webhooks asynchronously and update its internal knowledge base when a developer changes a ticket's status or description so that the AI Triage agent always has the most accurate, real-time context for detecting duplicates.

Epic 6: AI Analytics Dashboard

#	User Story
1	As a Product Owner, I want to view aggregated daily customer sentiment trends (Positive, Neutral, Negative) on an interactive dashboard so that I can instantly gauge the overall reception of our latest product release.
2	As a Product Owner, I want to see an AI-generated summary of the most requested features and most frequent bugs so that I can make data-driven decisions during sprint planning.

Epic 7: AI Product Assistant (Conversational Memory)

#	User Story
1	As a Product Owner, I want to query my entire synchronized backlog using natural language via an AI chat interface (e.g., "What are the most requested payment features this month?") so that I can instantly discover deep trends without writing complex SQL or JQL queries.
2	As a Product Owner, I want the AI Assistant to save and remember my conversational history within a specific session so that I can ask follow-up questions and drill down into the context of specific customer issues seamlessly.

Epic 8: Tenant Registration & SaaS Operations

#	User Story
1	As a Product Owner, I want to self-register my company on the FeedInsight platform so that we can automatically generate our Tenant profile and begin configuring our workspace and Jira credentials.
2	As a Super Admin, I want to view a comprehensive list of all self-registered companies so that I can monitor which organizations are currently using the platform.
3	As a Super Admin, I want to easily toggle the status of any registered company (activate, suspend, or deactivate) so that I can restrict or grant platform access based on their subscription status or compliance.
4	As a Super Admin, I want to view a centralized directory of all Product Owners mapped across different tenants so that I can assist with platform troubleshooting or account locking if a client requests it.

Epic 9: Customer Portal & Customer Management

#	User Story
1	As a Company Product Owner, I want to create and manage customer accounts from my dashboard so that I can grant authorized customers access to submit feedback through the platform.
2	As a Company Customer, I want to access a dedicated portal and submit product feedback directly through the UI so that I can easily report my issues or ideas to the company's product team.
3	As a Company Customer, I want to view my submitted feedback along with replies and comments from the company's Product Owners so that I can track my requests and follow the discussion with the product team.

Epic 10: Global AI Telemetry & Cost Management (Langfuse Mirroring)

#	User Story
1	As a Super Admin, I want to monitor real-time token consumption (prompt and completion tokens) aggregated across all tenants so that I can track our global OpenAI API resource usage.
2	As a Super Admin, I want to see a breakdown of AI operational costs filtered by specific companies so that I can identify high-volume tenants and ensure our pricing tiers cover their actual API consumption.
3	As a Super Admin, I want a centralized dashboard tracking Langfuse metrics like average AI execution duration and error rates so that I can be alerted instantly if the Router or Triage agents face performance degradation or API failures.